Langage C#

Mme. Fatima-Ezzahra AIT BENNACER
f.aitbennacer@emsi.ma

2026 - 2027

Plan du cours

1. Introduction
2. Les Variables
3. Les instructions conditionnelles
4. Structures itératives
5. Les tableaux
6. Les listes
7. Les méthodes

Introduction au langage C#

- Langage de programmation moderne qui combine la programmation orientée objet, fonctionnelle, et générique
- C# : se présente comme la suite des langages C et C++, très proche de JAVA.
- Langage complètement orienté objet → pas de possibilité d'écrire des fonctions en dehors des classes, pas de variables globales.
- Crée par Microsoft en 2002, C# a évolué version par version : C#1.0 à C#14
- Conçu pour fonctionner avec .NET Framework, .NET core .

Les Bases du Langages C#

Les Variables

Les variables

- Des blocs de mémoire.
- Permettent de stocker une ou plusieurs données.
- Peuvent avoir plusieurs valeurs différentes dans un programme.
- Permettent d'effectuer:
  - Des calculs mathématiques.
  - Enregistrer l'âge du visiteur.
  - Comparer des valeurs

Types et déclaration

- Déclarer une variable:

```
type nomVariable;
```

- Les principaux types de base du framework .NET sont:

Type	Description
byte	Entier de 0 à 255
short	Entier de -32768 à 32767
int	Entier de -2147483648 à 2147483647
long	Entier de -9223372036854775808 à 9223372036854775807
float	Nombre simple précision de -3,402823e38 à 3,402823e38
double	Nombre double précision de -1,7976931348623e308 à 1,7976931348623e308
decimal	Nombre décimal convenant particulièrement aux calculs financiers (en raison de ses nombres significatifs après la virgule)
char	Représente un caractère
string	Une chaîne de caractère
bool	Une valeur booléenne (vrai ou faux)

Les mots réservés en C#

abstract	event	new	struct
as	explicit	null	switch
base	extern	object	this
bool	false	operator	throw
break	finally	out	true
byte	fixed	override	try
case	float	params	typeof
catch	for	private	uint
char	foreach	protected	ulong
checked	goto	public	unchecked
class	if	readonly	unsafe
const	implicit	ref	ushort
continue	in	return	using
decimal	int	sbyte	virtual
default	interface	sealed	volatile
delegate	internal	short	void
do	is	sizeof	while
double	lock	stackalloc	
else	long	static	
enum	namespace	string	

Types et déclaration

- Déclarer une variable:

```
int n;
```

- Déclarer et initialiser une variable:

```
int n = 5;
```

- Les conversions entre nombres et chaînes de caractères

nombre → chaine: ***nombre.ToString()***
chaine → int: ***int.Parse(chaine)***
chaine → long: ***long.Parse(chaine)***
chaine → double: ***double.Parse(chaine)***
chaine → float: ***float.Parse(chaine)***

Opérations de lecture et écriture

- Pour lire une chaîne saisie dans la console et l'enregistrer dans une variable:

```
string n = Console.ReadLine();
```

- Pour afficher le contenu d'une variable dans la console:

```
Console.WriteLine("Chaine saisie: "+ n);
```

Ou

```
Console.WriteLine($"Chaine saisie: {n}");
```

Remarques

- *string est le type de la valeur saisie et sauvegardée dans la variable n.*
- *$ permet de remplacer les variables situées entre {} par leurs valeurs respectives*

Opérations sur les variables

- Pour les variables numériques (int, float...)

= → affectation
+ → addition
- → soustraction
** → multiplication*
/ → division
% → reste de la division

Opérations sur les variables

- Pour les variables numériques (int, float...)

```
int a = 5; int b = 2;
Console.WriteLine($"a + b");
Console.WriteLine($"a - b");
Console.WriteLine($"a * b");
Console.WriteLine($"{a/b}");
Console.WriteLine($"(double)a/b");
Console.WriteLine($"{a%b}");
```

Opérations sur les variables

- Opérateurs particuliers:

i = i + 1 → i + +;
i = i - 1 → i - -;
i = i + 2 → i+ = 2;
i = i - 2 → i- = 2;

Opérations sur les variables

- L'opérateur + pour concaténer les chaînes de caractères:

```
string codePostal = ”1500”;
string ville = ”Rabat”;
string pays = ”Maroc”;
string adresse = codePostal + ” ” + ville + ” ” + pays;
```

Les Instructions Conditionnelles

Les opérateurs de comparaison

- Une condition se construit grâce à des opérateurs de comparaison. Les plus courants sont:

Opérateur	Description
==	Egalité
!=	Différence
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal
&&	ET logique
||	OU logique
!	Négation

If

- Permet d'exécuter le code (...) si une condition est vraie

```
if (condition)
{
...
}
```

Exemple

```
decimal compteEnBanque = 300;

if (compteEnBanque >= 0)

    Console.WriteLine("Votre compte est créditeur");
```

If ... else

```
if (condition)
{...}
else
{...}
```

Exemple

```
int age = 17;

if (age >= 18)
{
    Console.WriteLine("Vous êtes majeur(e).");
}
else
{
    Console.WriteLine("Vous êtes mineur(e).");
}
```

If ... else if ... else

On peut enchaîner les conditions avec else if

```
if (condition1)
{
...
}
   else if (condition2)
{
...
}
...
   else
{
...
}
```

If ... else if ... else

Exemple:

```
int score = 75;

if (score >= 90)
{
    Console.WriteLine("Excellent !");
}
else if (score >= 75)
{
    Console.WriteLine("Très bien !");
}
else if (score >= 50)
{
    Console.WriteLine("Assez bien");
}
else {    Console.WriteLine("Échec"); }
```

switch

- L'instruction switch peut être utilisée lorsqu'une variable peut prendre beaucoup de valeurs.

```
switch (expression){
case x :
      ...
break;

case y :
     ...
break;
default :
...
break;
}
```

switch

- L'instruction switch peut être utilisée lorsqu'une variable peut prendre beaucoup de valeurs.

```
int jour = 3;
switch (jour) {
    case 1:     Console.WriteLine("Lundi");       break;
    case 2:     Console.WriteLine("Mardi");       break;
    case 3:     Console.WriteLine("Mercredi");      break;
    case 4:     Console.WriteLine("Jeudi");       break;
    case 5:     Console.WriteLine("Vendredi");      break;
    case 6:     Console.WriteLine("Samedi");      break;
    case 7:     Console.WriteLine("Dimanche");      break;
    default:    Console.WriteLine("Numéro de jour invalide !");
       break;
}
```

switch

Remarques

- Le bloc default peut apparaître à n'importe quelle position dans switch. Quelle que soit sa position, il est toujours évalué en dernier, une fois que tous les cas ont été évaluées.
- En l'absence d'un bloc default et si aucun bloc case n'est exécuté, le bloc switch sera traversé sans être exécuté.
- break permet de quitter switch.
- Même dans bloc default, il faut placer un break.

Les Instructions Itératives

La boucle while

- Boucle while : à chaque itération on teste si la condition est vraie avant d'accéder aux traitements

```
while (condition)
{
    ...
}
```

Exemple

```
int i = 0;
while(i < 5){
    Console.WriteLine(i );
    i++;
}
```

La boucle do ... while

- La Boucle do…while exécute le bloc au moins une fois ensuite elle vérifie la condition:

```
do {
    ...
}
while(condition);
```

Exemple

```
int i = 0;
do{
    Console.WriteLine(i );
    i++;
}
while(i < 5);
```

La boucle for

- Elle permet de répéter un bout de code tant qu'une condition est vraie.

```
for (initialisation; condition; incrémentation){
    ...
}
```

Exemple

```
for(int i = 0; i < 5; i + +) {

    Console.WriteLine(i);

  }
```

Les Tableaux

Les Tableaux

- Un tableau (array) est une collection d'éléments du même type, stockés de manière contiguë en mémoire.
- Tous les éléments d'un tableau partagent le même type de données (ex : int, string, double, etc.)

Les Tableaux

Déclaration

```
type[ ] nomTableau = new type[nbrElement];
```

Exemple

```
int[] tab = new int[6];
```

Remarques

- Tous les éléments du tableau sont initialisés à 0.
- tab[i]: permet d'accéder à l'élément d'indice i du tableau.
- Le premier élément d'un tableau est d'indice 0.
- On ne peut ni dépasser la taille initiale d'un tableau ni changer le type déclaré.

Les Tableaux

Déclaration et initialisation

```
int [ ] tab = new int [ ] {3, 5, 4, 9, 7, 1};
int [ ] tab = {3, 5, 4, 9, 7, 1};

Ou
int[] tab;
tab = new int[] {3, 5, 4, 9, 7, 1};
```

```
tab[6] = 10; → IndexOutOfRangeException
```

Les Tableaux

Parcourir un tableau avec la boucle for

```
for (int i = 0; i < tab.Length; i++) {

    Console.WriteLine(tab[i ]);
}
```

Parcourir un tableau avec foreach

```
    foreach (int n in tab) {

    Console.WriteLine(n);

    }
```

Les Tableaux

Déclaration d'un tableau à deux dimensions

```
type[ , ] nomTableau = new type[nbrLignes, nbrColonnes];
```

Déclaration et initialisation

```
int [, ] Matrice =
{
    {1, 2},
    {3, 4}
};
```

Les Tableaux

Parcourir un tableau à deux dimensions avec la boucle for

```
for (int i = 0; i < 2; i++)
   for (int j = 0; j < 2; j++)
        Console.WriteLine(tab[i, j]);
```

Parcourir un tableau à deux dimensions avec foreach

```
foreach (int n in Matrice) {
    Console.Write(n);
}
```

Les Listes

Les Listes

Il y a des différences fondamentales entre une liste (utilisant **List<T>**) et un tableau (tableau unidimensionnel):

- Un tableau a une taille fixe lors de sa création. Une fois que la taille d'un tableau est définie, elle ne peut pas être modifiée sans créer un nouveau tableau. Une liste est dynamique et peut être redimensionnée (ajout/suppression d'éléments) à tout moment.
- L'ajout d'éléments à un tableau existant (sans créer un nouveau tableau) est inefficace car il nécessite de copier les éléments existants. Les listes sont plus efficaces pour l'ajout et la suppression d'éléments car elles gèrent automatiquement le redimensionnement de manière transparente.

Les Listes

Déclaration et initialisation

```
List<double> maListe = new List<double>() {1.0, 2.0, 3.0, 4.0, 5.0};
```

Parcourir une liste avec for

```
for (int i = 0; i < maListe.Count; i++) {
Console.WriteLine(maListe[i]);
}
```

Parcourir une liste avec foreach

```
foreach (int n in maListe) {
Console.WriteLine(n);
}
```

Les Méthodes

Les Méthodes

Le but de la méthode est de factoriser du code afin d'éviter d'avoir à répéter sans arrêt le même code.

Exemple 1

```
public static void DireBonjour()
{
    Console.WriteLine("Bonjour !");
    Console.WriteLine("Bienvenue dans le cours de C#");
}
```

Les Méthodes

Exemple 2

```
public static void DireBonjour(string prenom, int age)
{
   Console.WriteLine("Bonjour " +prenom);
   Console.WriteLine("Vous avez " +age + "ans");
   Console.WriteLine("Bienvenue dans le monde merveilleux du C#");
}
```

Les Méthodes

Exemple 3

```
public static double Somme(int n)
{
double somme = 0;
      for(int i = 0; i <= n; i++) {
            somme+ = i;
           }
return somme;
}
```

TP N°1